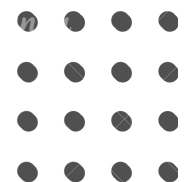




VMES Grinds





Leetcode: 2558
Take Gifts from the Richest Pile





2558. Take Gifts From the Richest Pile

Easy



Topics



Companies



Hint

You are given an integer array `gifts` denoting the number of gifts in various piles. Every second, you do the following:

- Choose the pile with the maximum number of gifts.
- If there is more than one pile with the maximum number of gifts, choose any.
- Reduce the number of gifts in the pile to the floor of the square root of the original number of gifts in the pile.

Return *the number of gifts remaining after* `k` *seconds.*



Example 1:

Input: `gifts = [25,64,9,4,100]`, `k = 4`

Output: 29

Explanation:

The gifts are taken in the following way:

- In the first second, the last pile is chosen and 10 gifts are left behind.
- Then the second pile is chosen and 8 gifts are left behind.
- After that the first pile is chosen and 5 gifts are left behind.
- Finally, the last pile is chosen again and 3 gifts are left behind.

The final remaining gifts are `[5,8,9,4,3]`, so the total number of gifts remaining is 29.

Constraints:

- `1 <= gifts.length <= 103`
- `1 <= gifts[i] <= 109`
- `1 <= k <= 103`

Example 2:

Input: `gifts = [1,1,1,1]`, `k = 4`

Output: 4

Explanation:

In this case, regardless which pile you choose, you have to leave behind 1 gift in each pile.

That is, you can't take any pile with you.

So, the total gifts remaining are 4.

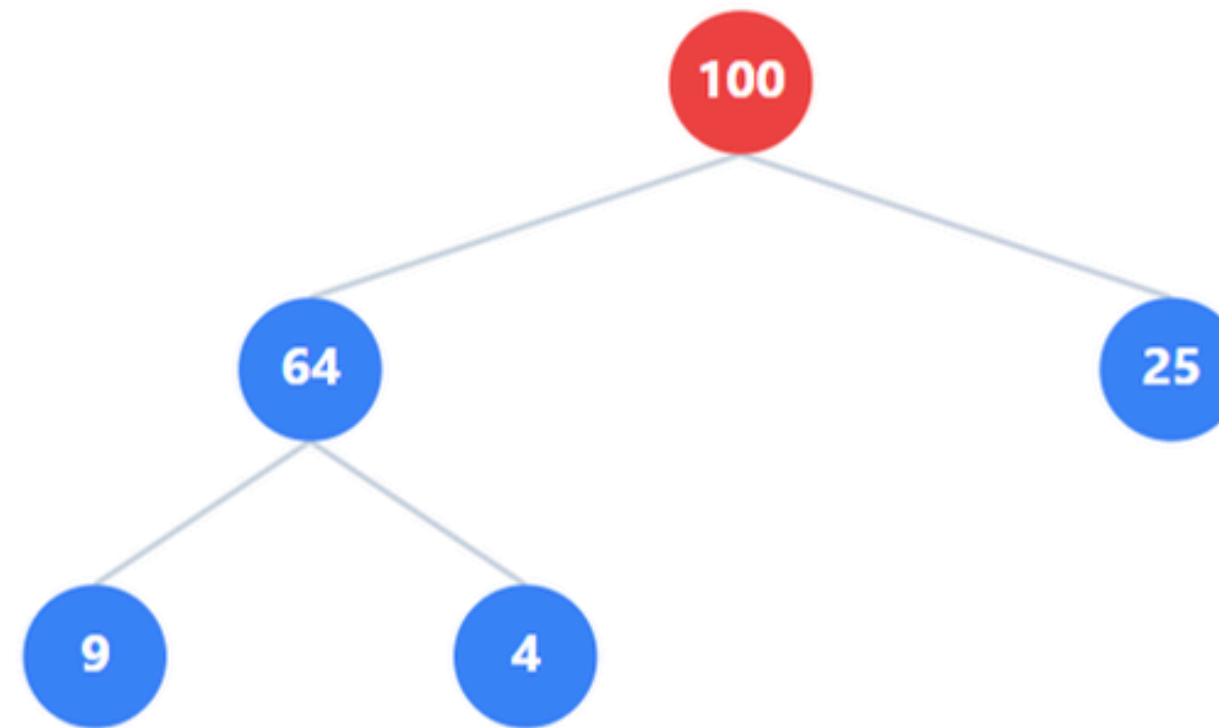


Explanation





Max-Heap Operations



Iteration 0 of 4 - Step 0

▶ Next

↺ Reset

Total Gifts: 202



Initial Approach





- 1. Find the largest element**
- 2. Take square root of that element**
- 3. Push it back to the list**
- 4. Do it for k times**
- 5. Sum the elements of the remaining list**
- 6. Done!**



1. Find the largest element
2. Take square root of that element
3. Push it back to the list
4. Do it for k times
5. Sum the elements of the remaining list
6. Done!

```
class Solution:
    def pickGifts(self, gifts: List[int], k: int) → int:
        for i in range(k):
            largest = max(gifts)
            gifts.remove(largest)
            gifts.append(int(math.isqrt(largest)))

        return sum(gifts)
```



Optimized Approach





- 1. Convert the list into heap!**
- 2. Pop out the first element**
- 3. Take square root of that element**
- 4. Push it back to the heap**
- 5. Do it for k times**
- 6. Sum the elements of the remaining list**
- 7. Done!**



1. Convert the list into heap!
2. Pop out the first element
3. Take square root of that element
4. Push it back to the heap
5. Do it for k times
6. Sum the elements of the remaining list
7. Done!

```
class Solution:
    def pickGifts(self, gifts: List[int], k: int) → int:
        negative = [-g for g in gifts]
        heapq.heapify(negative)

        for i in range(k):
            largest = -1 * heapq.heappop(negative)
            reduced = int(math.isqrt(largest))
            heapq.heappush(negative, -1 * reduced)

        return -1 * sum(negative)
```



Time and Space Complexity Analysis





Space - $O(1)$

Time - $O(k * n)$

```
class Solution:
    def pickGifts(self, gifts: List[int], k: int) → int:
        for i in range(k):
            largest = max(gifts)
            gifts.remove(largest)
            gifts.append(int(math.isqrt(largest)))

        return sum(gifts)
```

$O(n)$



Space - $O(n)$

Time - $O(n + k \log n)$

```
class Solution:
    def pickGifts(self, gifts: List[int], k: int) → int:
        negative = [-g for g in gifts]
        heapq.heapify(negative)                                 $O(n)$ 

        for i in range(k):
            largest = -1 * heapq.heappop(negative)
            reduced = int(math.isqrt(largest))                  $O(\log n)$ 
            heapq.heappush(negative, -1 * reduced)

        return -1 * sum(negative)
```



Key Takeaway



1. Max-Heap gives efficient access to the largest pile in $O(\log n)$.
2. We can convert negating values to simulate a max-heap using `heapq`.